

Universidad Rafael Landívar

Quetzaltenango

Facultad de Ingeniería

Departamento de Informática y Sistemas

Estructura de Datos II

Ing. Manuel Gustavo Rojas Vásquez

MANUAL TÉCNICO

Jackelin Sofia Vásquez Guzmán Carné 1503725

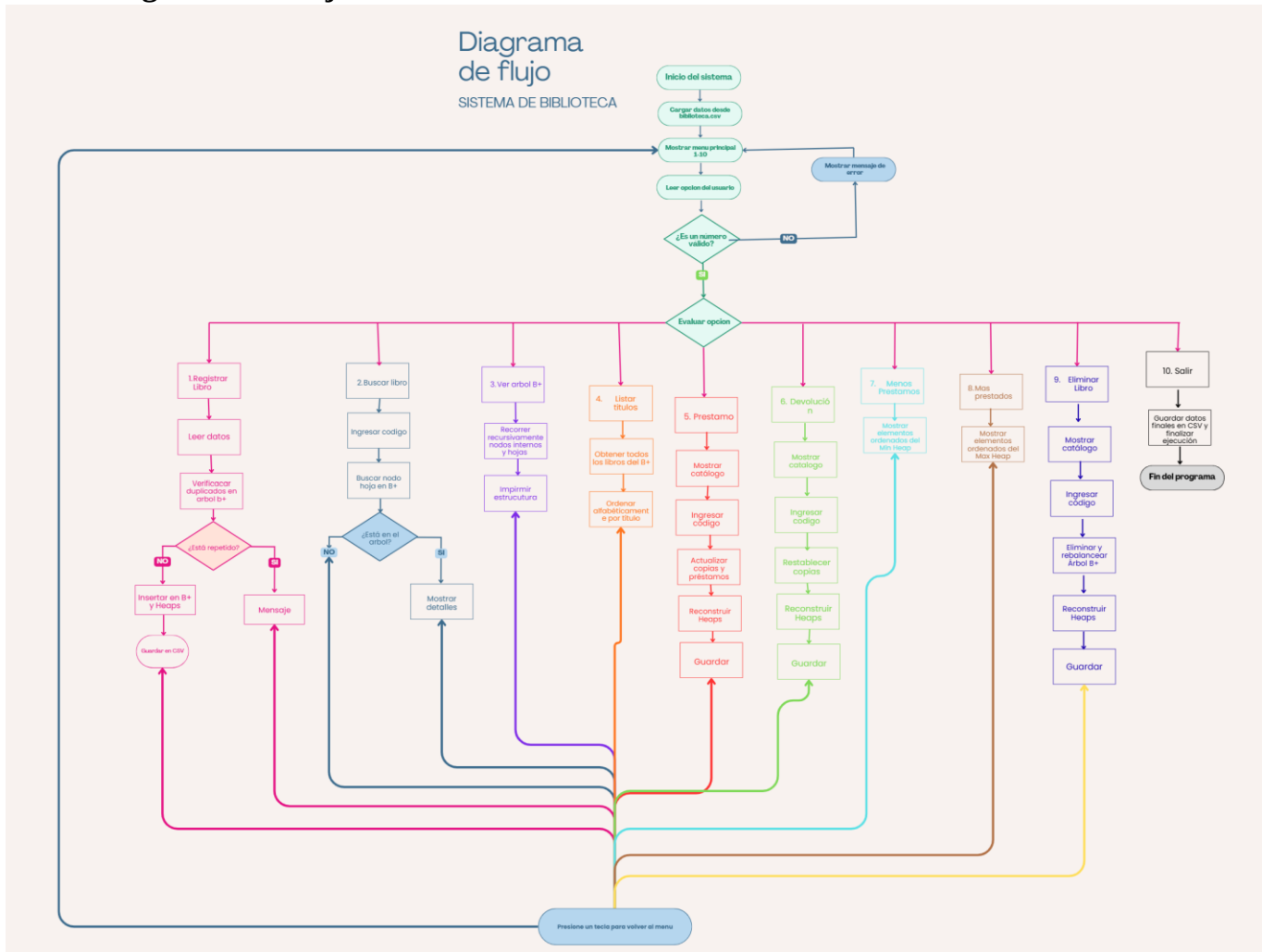
08 de Septiembre de 2026

Índice

Manual Técnico: Proyecto Catálogo de Biblioteca	2
1. Diagrama de Flujo	2
2. Diagrama de Clases	3
3. Explicación de Implementación y Justificación de Estructura de Datos	3
Explicación de la Implementación del Sistema	3
1. Implementación del Árbol B+ (ArbolPlus.cs y Nodo.cs)	3
2. Implementación del Min Heap y Max Heap (MinHeap.cs y MaxHeap.cs)	4
3. Persistencia y manejo del archivo CSV (Program.cs)	4
Justificación de Estructuras de Datos	5
Árbol B+:	5
Min y Max Heap:	6

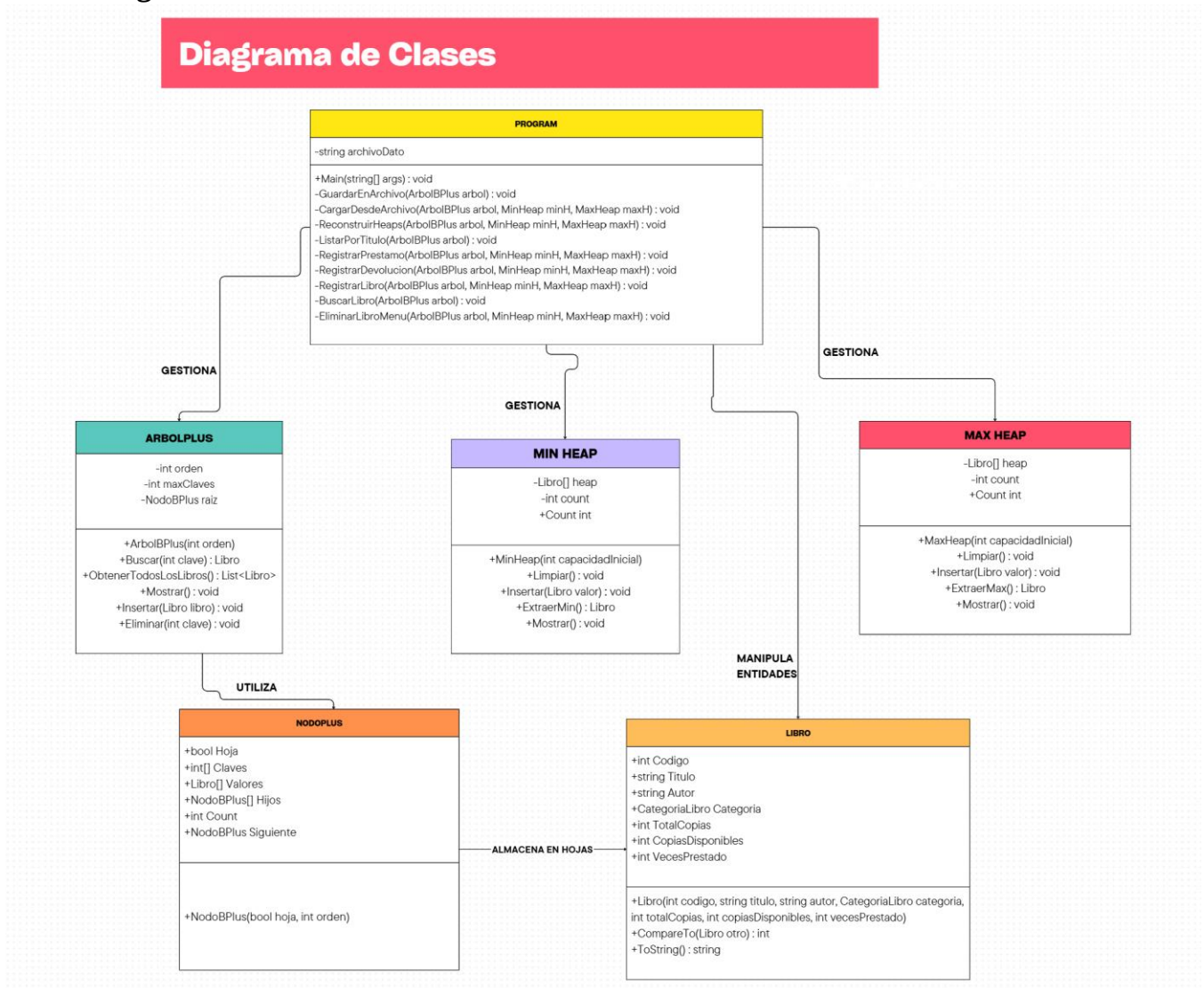
Manual Técnico: Proyecto Catálogo de Biblioteca

1. Diagrama de Flujo



Link de diagrama: <https://canva.link/udrt90s7whu96w4>

2. Diagrama de Clases



3. Explicación de Implementación y Justificación de Estructura de Datos

Explicación de la Implementación del Sistema

El proyecto fue desarrollado completamente en C#, utilizando un diseño modular y orientado a objetos. La implementación de las estructuras de datos se realizó de forma manual, utilizan arreglos y referencias entre nodos, además de métodos propios para realizar las operaciones de inserción, búsqueda, eliminación y reorganización.

1. Implementación del Árbol B+ (ArbolPlus.cs y Nodo.cs)

El Árbol B+ está formado por diferentes nodos que almacenan las claves, los libros y, dependiendo del tipo de nodo, las referencias hacia otros nodos.

- **Estructura de los nodos (NodoBPlus):** Cada nodo utiliza arreglos para almacenar las claves y los libros. Los nodos internos también cuentan con un arreglo de referencias a sus hijos. Además, los nodos hoja tienen una referencia Siguiente, que permite conectarlos con la siguiente hoja.
- **Búsqueda:** La búsqueda comienza desde la raíz. El programa compara el código que se está buscando con las claves del nodo actual para determinar hacia qué hijo debe continuar. Este proceso se repite hasta llegar a una hoja, donde se comprueba si el código existe y se obtiene el libro correspondiente.
- **Inserción y división (Split):** Para agregar un libro, primero se localiza la hoja donde debe colocarse. La nueva clave se inserta manteniendo el orden de las claves existentes. Cuando el nodo supera la cantidad máxima permitida, se realiza una división. En las hojas, los elementos se distribuyen entre el nodo original y un nuevo nodo, manteniendo también el enlace mediante Siguiente. Si la división corresponde a un nodo interno, sus claves e hijos se distribuyen de forma similar y se actualiza el nodo padre cuando es necesario.
- **Eliminación y reorganización:** Para eliminar un libro, se busca primero su código dentro de las hojas. Al encontrarlo, los elementos posteriores se desplazan para ocupar el espacio que queda disponible. Si la eliminación provoca que el nodo tenga menos elementos de los permitidos, se revisan sus hermanos para determinar si es posible redistribuir elementos o si es necesario realizar una fusión.

2. Implementación del Min Heap y Max Heap (MinHeap.cs y MaxHeap.cs)

Los dos montículos utilizan un arreglo de tipo Libro[] para almacenar los elementos. La diferencia principal entre ambas clases está en la forma en que realizan las comparaciones: el Min Heap coloca en la raíz el libro con menor cantidad de préstamos, mientras que el Max Heap coloca el de mayor cantidad.

- **Almacenamiento:** Se utiliza un arreglo junto con una variable que indica cuántos elementos contiene actualmente. Cuando el arreglo alcanza su capacidad máxima, se crea uno nuevo con mayor tamaño y se copian manualmente los elementos existentes.
- **Inserción (HeapifyUp):** Al insertar un libro, este se coloca al final del arreglo. Después, se compara con su padre y se intercambian las posiciones mientras sea necesario para mantener la propiedad correspondiente del montículo. La posición del padre se obtiene mediante $(i-1)/2$.
- **Eliminación (HeapifyDown):** Cuando se elimina el elemento de la raíz, se coloca en su lugar el último elemento del arreglo. Luego se compara con sus hijos y se realizan los intercambios necesarios para volver a establecer la propiedad del Min Heap o Max Heap. Las posiciones de los hijos se calculan mediante $2i+1$ y $2i+2$.

3. Persistencia y manejo del archivo CSV (Program.cs)

El sistema utiliza un archivo .csv para almacenar los datos de los libros y evitar que estos tengan que estar escritos directamente en el código.

- **Lectura del archivo:** Al cargar los datos, el programa utiliza `File.ReadAllLines` para obtener las líneas del archivo. Cada línea corresponde a un libro registrado.
- **Procesamiento de los datos:** Las líneas se separan mediante `.Split(',')` para obtener individualmente el código, título, autor, categoría, cantidad de copias y cantidad de préstamos. Los valores obtenidos se convierten posteriormente al tipo de dato correspondiente.
- **Escritura y actualización:** Cuando se realiza una operación que modifica la información de un libro, como un registro, préstamo, devolución o eliminación, los datos se actualizan y posteriormente se guardan nuevamente en el archivo mediante `StreamWriter`.

De esta manera, el programa mantiene separadas las responsabilidades de cada parte: el Árbol B+ se encarga de manejar sus propios nodos y operaciones, los montículos administran los libros según su cantidad de préstamos y `Program.cs` se encarga principalmente de la interacción con el usuario y del manejo de los archivos.

Justificación de Estructuras de Datos

Árbol B+:

El árbol B+ se seleccionó como la estructura principal para indexar y organizar el catálogo de libros, principalmente porque permite realizar búsquedas, inserciones y eliminaciones de manera eficiente, manteniendo los datos organizados y el árbol equilibrado. Estas operaciones tienen una complejidad tipo ($O(\log n)$), por lo que el tiempo de búsqueda no aumenta de una forma excesiva a medida que crece la cantidad de libros que son registrados.

En el sistema cada libro cuenta con un código único, por lo que este código puede utilizarse como clave dentro del Árbol B+. De esta manera, cuando el usuario necesita consultar un libro específico, no es necesario revisar todos los registros uno por uno. El árbol permite ir comparando la clave buscada y decidir qué parte de la estructura debe recorrerse hasta encontrar el libro que se desea. Esto resulta muy útil para una biblioteca, ya que el catálogo puede llegar a contener una cantidad muy grande de registros.

Otra característica importante por la que se seleccionó esta estructura es que en el Árbol B+ los datos se almacenan en los nodos hoja y estos se encuentran enlazados entre sí. Esta organización resulta beneficiosa para el proyecto porque permite mantener los libros organizados según su código y realizar recorridos secuenciales entre los diferentes registros. Además, los enlaces entre las hojas facilitan el acceso consecutivo a los libros almacenados en el catálogo.

El sistema también cuenta con una opción para generar un listado completo del catálogo ordenado por título. Para realizar esta operación, primero se obtienen los libros almacenados en el Árbol B+ y posteriormente se realiza un ordenamiento tomando como criterio el título del libro. De esta manera, el Árbol B+ se utiliza principalmente para indexar y buscar los libros mediante su código, mientras que el ordenamiento por título se realiza como un proceso adicional cuando el usuario selecciona esta opción.

También se considera adecuado para las operaciones de inserción y eliminación, ya que el Árbol B+ mantiene su estructura equilibrada cuando se agregan o eliminan registros. Esto permite que las operaciones posteriores continúen siendo eficientes y evita que la estructura se deforme como podría ocurrir con un árbol BST en determinadas situaciones debido al desequilibrio que puede ocurrir en este.

Por estas razones, el Árbol B+ se utiliza como estructura principal dentro del catálogo, ya que el código del libro permite localizar rápidamente un registro, mientras que la organización de sus nodos hoja facilita el recorrido ordenado de los libros según su código. Además, el sistema permite generar de forma independiente el listado del catálogo ordenado por título. De esta forma, la estructura se adapta a las principales operaciones que requiere el sistema, como registrar libros, buscar libros por código, eliminar registros y consultar el catálogo ordenado por título, incluso cuando la cantidad de registros aumenta.

Min y Max Heap:

El Max y Min Heap se implementaron para gestionar los libros según la cantidad de veces que han sido prestados. Se utilizan dos estructuras diferentes porque cada una permite obtener de forma eficiente un extremo distinto de los datos, por un lado el Min Heap permite identificar rápidamente el libro con menor cantidad de préstamos debido a la prioridad que maneja (el dato mínimo es el que se prioriza), mientras que el Max Heap permite identificar el libro con mayor cantidad de préstamos ya que prioriza el dato mayor.

El Max Heap resulta especialmente útil para generar reportes de los libros más prestados de la biblioteca. Al mantener el libro con mayor número de préstamos en la raíz del montículo, se puede acceder directamente al elemento de mayor prioridad sin tener que ordenar todos los libros cada vez que se solicita este tipo de información. Esto permite identificar de manera rápida cuáles son los libros más solicitados y puede utilizarse para generar un listado de los libros con mayor cantidad de préstamos.

Por otro lado, el Min Heap permite realizar la operación contraria, ya que mantiene en la raíz el libro que tiene la menor cantidad de préstamos. Esto permite identificar rápidamente los libros que han sido menos solicitados. Esta información es útil para reportes y analizar qué libros tienen una menor demanda dentro de la biblioteca.

Una de las principales ventajas de los montículos es que permiten consultar el elemento mínimo o máximo que se encuentra en la raíz por el tipo de complejidad que maneja que es $O(1)$. Sin embargo, cuando se agregan o eliminan elementos, es necesario mantener la propiedad del montículo para que el elemento correspondiente continúe ocupando la posición correcta. Para ello dentro de la estructura se manejan las operaciones de HeapifyUp y HeapifyDown, que permiten reorganizar los elementos del montículo en un tiempo $O(\log n)$.

Por ello en este proyecto, los montículos se utilizan de forma complementaria al Árbol B+, mientras que el árbol b+ se encarga principalmente de indexar los libros y permitir su búsqueda mediante el código, los montículos se enfocan en organizar los libros de acuerdo

con la cantidad de préstamos. De esta manera, cada estructura cumple una función específica dentro del sistema y se aprovechan sus características para realizar de forma más eficiente las operaciones que requiere el catálogo de la biblioteca.